COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 / Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	08/17/2026	**DATE SUBMITTED:**	09/03/2026
NAME:	Lance Cristopher S. Salcedo	**INSTRUCTOR:**	Engr. Jimlord Quejardo

HANDS ON ACTIVITY NO. 6.1 **Searching Techniques**

1. PROCEDURE OUTPUT

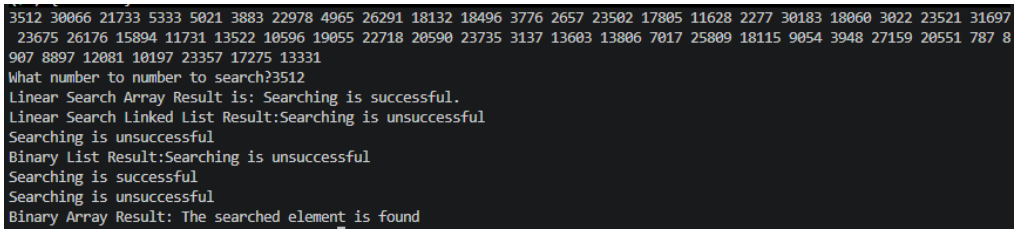
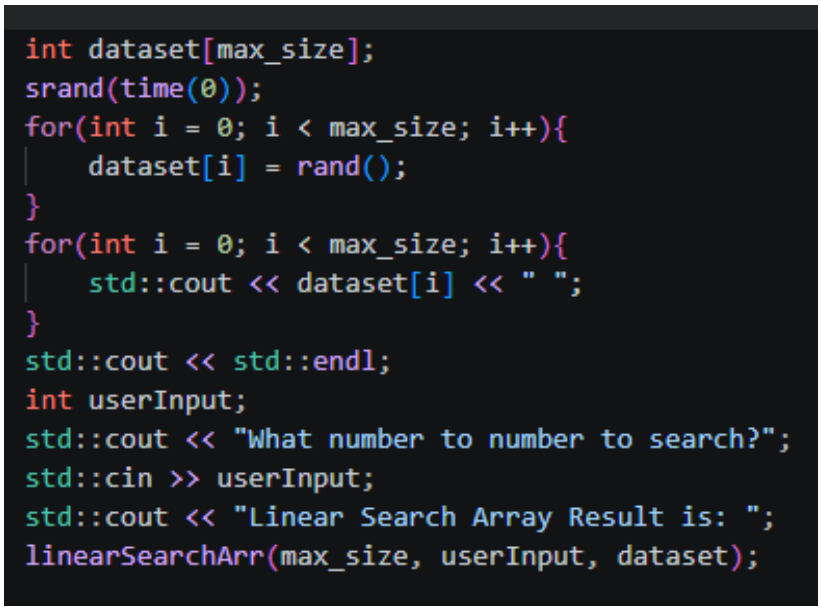
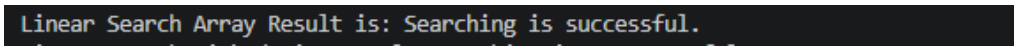
Screenshot	
Observations	From what I observed, I have to find, select or type a number to search. For Linear Search Array it was successful, the Linear Search Linked List is unsuccessful, Binary Search Array is successful and Binary Search Linked List somehow is both successful and unsuccessful..

Table 6.1 Data Generated and Observations

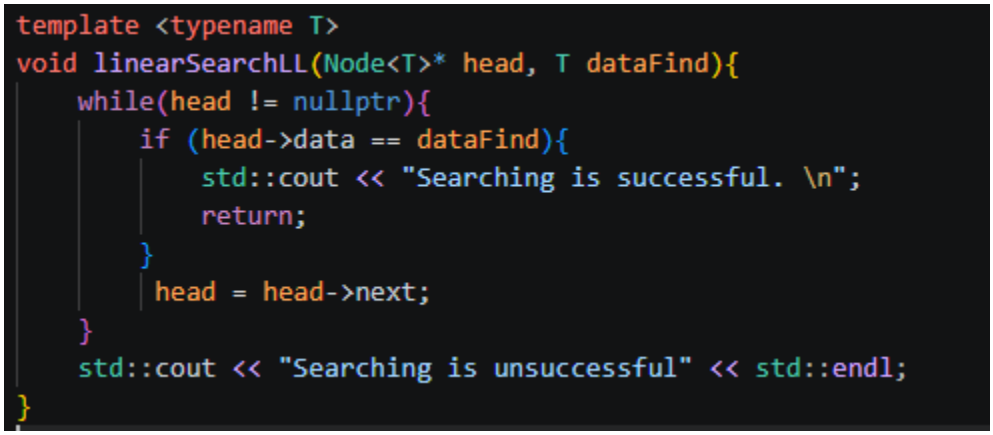
Code	```
	template <typename T>
	void linearSearchArr(int arrSize, T dataFind, T list []){
	    int i = 0;
	    while(i < arrSize){
	        if (dataFind == list[i]){
	            std::cout << "Searching is successful. \n";
	            return;
	        }
	        i++;
	    }
	    std::cout << "Searching is unsuccessful" << std::endl;
	}
	``
	Main.cpp:

	
Output	
Observations	For the linear search array, in the code that was given, in the h file, the code will go into a while loop if the int i is less than the array size. In the while loop if both the dataFind and list[i] matched it will return. Then in the main file, the code uses srand(time(0)) to create random numbers that create the dataset. When we put the chosen number from any of the random numbers that was generated by srand(time(0)) it will trigger the linearSearchArr function.

```
int dataset[max_size];
srand(time(0));
for(int i = 0; i < max_size; i++){
    dataset[i] = rand();
}
for(int i = 0; i < max_size; i++){
    std::cout << dataset[i] << " ";
}
std::cout << std::endl;
int userInput;
std::cout << "What number to number to search?";
std::cin >> userInput;
std::cout << "Linear Search Array Result is: ";
linearSearchArr(max_size, userInput, dataset);
```

```
Linear Search Array Result is: Searching is successful.
```

Table 6.2a Linear Search for Arrays

Code	

```
template <typename T>
void linearSearchLL(Node<T>* head, T dataFind){
    while(head != nullptr){
        if (head->data == dataFind){
            std::cout << "Searching is successful. \n";
            return;
        }
        head = head->next;
    }
    std::cout << "Searching is unsuccessful" << std::endl;
}
```

main.cpp:

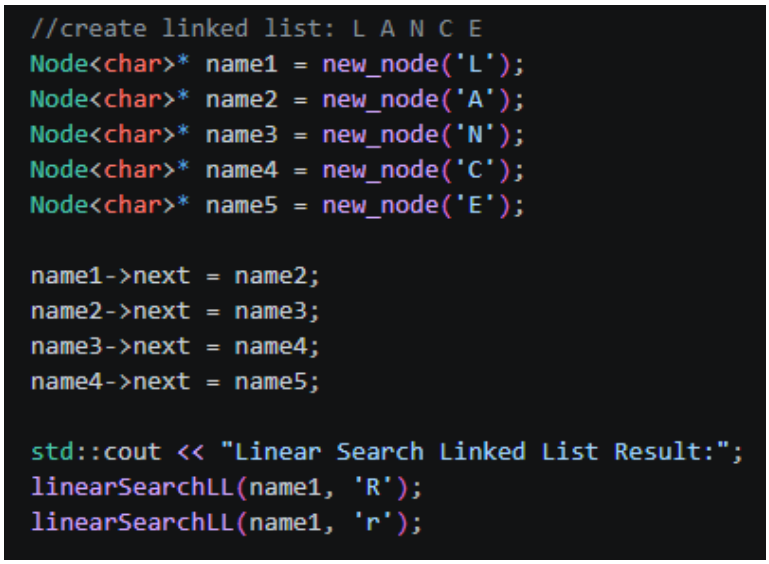
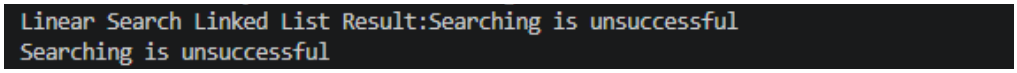
	
Output	
Observations	For the linear search linked list, in the h file, the code that was given has a node for the linked list to follow. Similarly to the linear search array, it's the same but it's for linked lists and needs a node to make it work. However, it needs the head->data and the nullptr to be the same to return. In the main file, we create a heap memory of 5 of my names then link each node together. When the linearSearchLL function was called it was unsuccessful since none of the letters in the memory was there to search.

Table 6.2b Linear Search for Linked List

Code

```
template <typename T>
void binarySearchArr(int arrSize, T dataFind, T list[]){
    // define low, high and middle
    int low = 0, high = arrSize-1, middle;
    //loop while is less than or equal to high
    while(low <= high){
        //compute for middle
        middle = (int)((low+high)/2);
        //if data found is middle -> data found -> return
        if (dataFind == list[middle]){
            std::cout << "The searched element is found \n";
            return;
        }
        //if data find < data middle -> new high
        else if (dataFind < list[middle]){
            high = middle -1;
        }else{
            //else update low = middle +1
            low = middle +1;
        }
    }
    //search is not found
    std::cout << "Searched element is not found" << std::endl;
}
```

main.cpp:

```
std::cout << "Binary Array Result: ";
binarySearchArr(max_size, userInput, dataset);
```

Output

```
Binary Array Result: Searched element is not found
```

Observations

For the binary search array, in the h file, the code first defines the low, high and middle, the code also makes a computation for the middle in the while loop. Inside of the while loop, the code has three if/else statements for each scenario. If the data was found it returns, if the data was less than compared to the middle and if the low is equal to the middle. Back to the main file, when the function is called the result is somehow not found.

Table 6.3a Binary Search for Arrays

Code

```
template <typename T>
void binarySearchLL(Node<T>* start, T dataFind){
    Node<T>* last = nullptr;

    while(start != last){

    //find middle
    Node<T>* middle = getMiddle(start, last);
    // middle contains value
    if(middle->data == dataFind){
        std::cout << "Searching is successful\n";
        return;
    }
    //search left
    else if(middle->data > dataFind){
        last = middle;
    }
    //search right
    else{
        start = middle->next;
    }
}
    std::cout << "Searching is unsuccessful\n";
}
```

getMiddle:

```
template <typename T>
Node<T> *getMiddle(Node<T>* start, Node<T>* last){

    Node<T>* slow = start;
    Node<T>* fast = start;

    while(fast != last && fast->next != last){
        //move slow one step
        slow = slow->next;
        //move fast two steps
        fast = fast->next->next;
    }
    return slow;
}
```

Main.cpp:

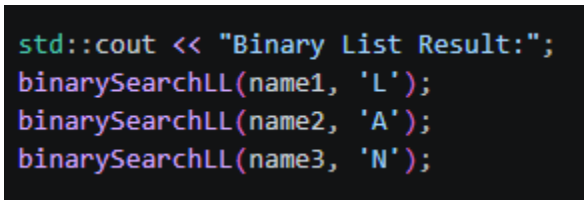
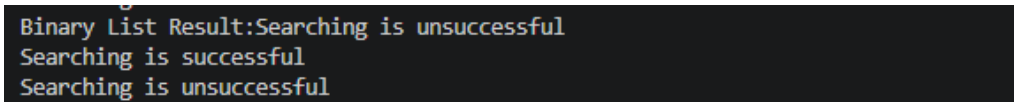
	
Output	
Observations	For the binary search linked list, In the h file, it is similar to the array but needs a node to function it, the code given also has a getMiddle function to get the middle. It also checks if the node is empty or nullptr before checking the node. Similar to the binary search array, it also has three if/else statements, if the middle has now the data it returns, searching left and searching right. Then in the main file, we put three letters in the heap memory. When the binarySearchLL was called it searches the data that was in the middle, the data detected in the middle is A and the search was successful while the other two letters L, N were not in the middle so it says it's not successful.

Table 6.3b Binary Search for Linked List

Analy

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem	Problem 1. Suppose you are doing a sequential search of the list [15, 18, 2, 19, 18, 0, 8, 14, 19, 14]. Utilizing both a linked list and an array approach to the list, use sequential search and identify how many comparisons would be necessary to find the key '18'?
Code	Array: supplementaryAlgo.h:

```cpp
template <typename A>
void suppleLinearSearchArr(int arrSize, A dataFind, A list []){
    int i = 0;
    int ArrNum = 0;
    while(i < arrSize){
        if (dataFind == list[i]){
            ArrNum++;
            std::cout << "Searching is successful. \n";
            std::cout << "Number key is found. \n";
            std::cout << "Number of comparisons: " << ArrNum << std::endl;
            return;
        }
        i++;
        ArrNum++;
    }
    std::cout << "Searching is unsuccessful \n";
    std::cout << "Number is not found." << std::endl;
};
```

supplementaryAlgo.cpp:

```cpp
//Problem 1 - Array
const int size = 10;
int Arr[size] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
std::cout << "Find 18 in this array. \n";
std::cout << "Linear Search Array Result:\n";
suppleLinearSearchArr(size, 18, Arr);
```

Node:

```cpp
#ifndef NODE2_H
#define NODE2_H
#include <iostream>

class Node2{
    public:
        int data;
        Node2 *next;
};

Node2 *new_Node(int newData){
    Node2 *newNode = new Node2;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

```

Linked List:

supplementaryAlgo.h:

```cpp
template <typename A>
void suppleLinearSearchLL(Node2* head, A dataFind){
    int LLNum = 0;
    while(head != nullptr){
        LLNum++;
        if (head->data == dataFind){
            std::cout << "Searching is successful. \n";
            std::cout << "Number key is found. \n";
            std::cout << "Number of comparisons: " << LLNum << std::endl;
            return;
        }
        head = head->next;
    }
    std::cout << "Searching is unsuccessful. \n";
    std::cout << "Number is not found." << std::endl;
};
```

supplementaryAlgo.cpp:

```
//Problem 1 - Linked List
Node2* Num1 = new_Node(15);
Node2* Num2 = new_Node(18);
Node2* Num3 = new_Node(2);
Node2* Num4 = new_Node(19);
Node2* Num5 = new_Node(18);
Node2* Num6 = new_Node(0);
Node2* Num7 = new_Node(8);
Node2* Num8 = new_Node(14);
Node2* Num9 = new_Node(19);
Node2* Num10 = new_Node(14);

Num1->next = Num2;
Num2->next = Num3;
Num3->next = Num4;
Num4->next = Num5;
Num5->next = Num6;
Num6->next = Num7;
Num7->next = Num8;
Num8->next = Num9;
Num9->next = Num10;
Num10->next = nullptr;

std::cout << "Find 18 in this linked list. \n";
std::cout << "Linear Search Linked List Result:\n";
suppleLinearSearchLL(Num1, 18);
```

Output

Array:

```
Find 18 in this array.
Linear Search Array Result:
Searching is successful.
Number key is found.
Number of comparisons: 2
```

Linked List:

```
Find 18 in this linked list.
Linear Search Linked List Result:
Searching is successful.
Number key is found.
Number of comparisons: 2
```

Table 6.4a Problem #1

Problem	Problem 2. Modify your sequential search algorithm so that it returns the count of repeating instances for a given search element 'k'. Test on the same list given in problem 1.

Code

Array:

supplementaryAlgo.h:

```
template <typename A>
void suppleLinearSearchArrIns(int arrSize, A dataFind, A list []){
    int i = 0;
    int ArrIns = 0;
    while(i < arrSize){
        if (dataFind == list[i]){
            ArrIns++;
        }
        i++;
    }
    std::cout << "Instances of: " << dataFind << ArrIns << std::endl;
};
```

supplementaryAlgo.cpp:

```
//Problem 2 - Array
int k;
std::cout << "------------" << std::endl;
std::cout << "Problem 2" << std::endl;
std::cout << "Array & Linked List Instances Count\n";
std::cout << "Enter number of instances: ";
std::cin >> k;
std::cout << "Linear Search Array Result:\n";
suppleLinearSearchArrIns(size, k, Arr);
```

Node:

```cpp
#ifndef NODE2_H
#define NODE2_H
#include <iostream>

class Node2{
    public:
        int data;
        Node2 *next;
};

Node2 *new_Node(int newData){
    Node2 *newNode = new Node2;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

```

Linked List:

supplementaryAlgo.h:

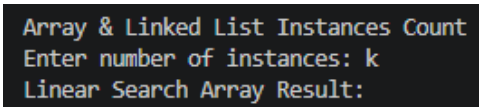
```cpp
template <typename A>
void suppleLinearSearchLLIns(Node2* head, A dataFind){
    int LLIns = 0;
    while(head != nullptr){
        if (head->data == dataFind){
            LLIns++;
        }
        head = head->next;
    }
    std::cout << "Instances of: " << dataFind << LLIns << std::endl;
};
#endif
```

supplementaryAlgo.cpp:

```cpp
//Problem 2 - Linked List
std::cout << "Linear Search Linked List Result:\n";
suppleLinearSearchLLIns(Num1, k);
```

Output

Array:	Linked List:



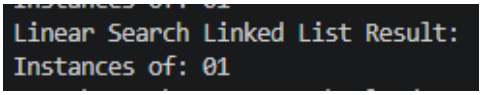


Table 6.4b Problem #2

Problem	Problem 3. Suppose you have the following sorted list [3, 5, 6, 8, 11, 12, 14, 15, 17, 18] and are using the binary search algorithm. If you wanted to find the key 8, draw a diagram that shows how the searching works per iteration of the algorithm. Prove that your drawing is correct by implementing the algorithm and showing a screenshot of the code and the output console.

Code	Array:
	supplementaryAlgo.h:
	`template <typename A>`
	`void suppleBinarySearchArrFind(int arrSize, A dataFind, A list[]){`
	`  int low = 0;`
	`  int high = arrSize-1;`
	`  int middle;`
	`  int BSArr = 0;`
	`  while(low <= high){`
	`  middle = (int)((low+high)/2);`
	`  if (dataFind == list[middle]){`
	`  BSArr++;`
	`  std::cout << "Finding element...\n";`
	`  std::cout << "Number: " << list[middle] << std::endl;`
	`  std::cout << "Number key is found. \n";`
	`  std::cout << "Number of comparisons: " << BSArr << std::endl;`
	`  std::cout << "Searching is successful. " << std::endl;`
	`  return;`
	`  }`
	`  else if (dataFind < list[middle]){`
	`  BSArr++;`
	`  std::cout << "Number: " << list[middle] << std::endl;`
	`  std::cout << "Wrong number." << std::endl;`
	`  std::cout << "Finding the right number..." << std::endl;`
	`  high = middle -1;`
	`  }else{`
	`  BSArr++;`
	`  std::cout << "Number: " << list[middle] << std::endl;`
	`  std::cout << "Wrong number." << std::endl;`
	`  std::cout << "Finding the right number..." << std::endl;`
	`  low = middle +1;`
	`  }`
	`  }`

```
    std::cout  << "Searched element is not found." << std::endl;
    std::cout  << "Searching is unsuccessful." << std::endl;
}
```

supplementaryAlgo.cpp:

```
//Problem 3 - Array
int BSArr[size] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
std::cout << "------------" << std::endl;
std::cout << "Problem 3" << std::endl;
std::cout << "Array: " << std::endl;
std::cout << "Finding the number 8." << std::endl;
std::cout << "Binary Search Array Result:" << std::endl;
suppleBinarySearchArrFind(size, 8, BSArr);
```

getMiddle:

```
Node2 *suppleMiddle(Node2* start, Node2* last){

    Node2* slow = start;
    Node2* fast = start;

    while(fast != last && fast->next != last){
        slow = slow->next;
        fast = fast->next->next;
    }
    return slow;
}
```

Linked List:

supplementaryAlgo.h:

```
template <typename A>
void suppleBinarySearchLLFind(Node2* start, A dataFind){
    Node2* last = nullptr;
    int BSLL = 0;
    while(start != last){
        Node2* middle = suppleMiddle(start, last);
        if(middle->data == dataFind){
            BSLL++;
            std::cout << "Finding element...\n";
            std::cout << "Number: " << middle->data << std::endl;
            std::cout << "Number key is found. \n";
            std::cout << "Number of comparisons: " << BSLL << std::endl;
            std::cout << "Searching is successful. " << std::endl;
            return;
        }
        else if(middle->data > dataFind){
            BSLL++;
            std::cout << "Number: " << middle->data << std::endl;
            std::cout << "Wrong number." << std::endl;
            std::cout << "Finding the right number..." << std::endl;
            last = middle;
        } else{
            BSLL++;
            std::cout << "Number: " << middle->data << std::endl;
            std::cout << "Wrong number." << std::endl;
            std::cout << "Finding the right number..." << std::endl;
            start = middle->next;
        }
    }
    std::cout  << "Searched element is not found." << std::endl;
    std::cout  << "Searching is unsuccessful." << std::endl;
};
```

supplementaryAlgo.cpp:

```
//Problem 3 - Linked List
std::cout << "------------" << std::endl;
std::cout << "Linked List:" << std::endl;

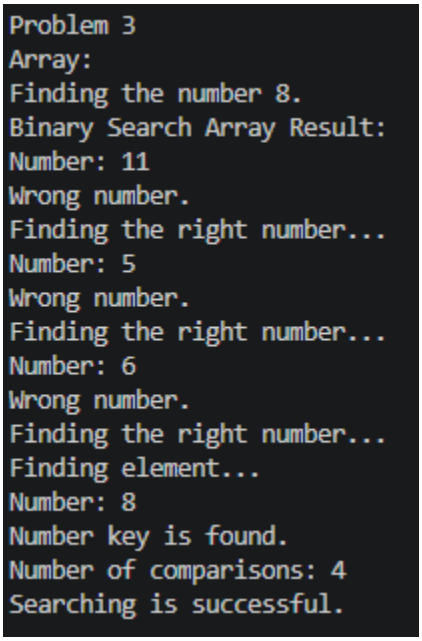
Node2* Num11 = new_Node(3);
Node2* Num12 = new_Node(5);
Node2* Num13 = new_Node(6);
Node2* Num14 = new_Node(8);
Node2* Num15 = new_Node(11);
Node2* Num16 = new_Node(12);
Node2* Num17 = new_Node(14);
Node2* Num18 = new_Node(15);
Node2* Num19 = new_Node(17);
Node2* Num20 = new_Node(18);

Num11->next = Num12;
Num12->next = Num13;
Num13->next = Num14;
Num14->next = Num15;
Num15->next = Num16;
Num16->next = Num17;
Num17->next = Num18;
Num18->next = Num19;
Num19->next = Num20;
Num20->next = nullptr;

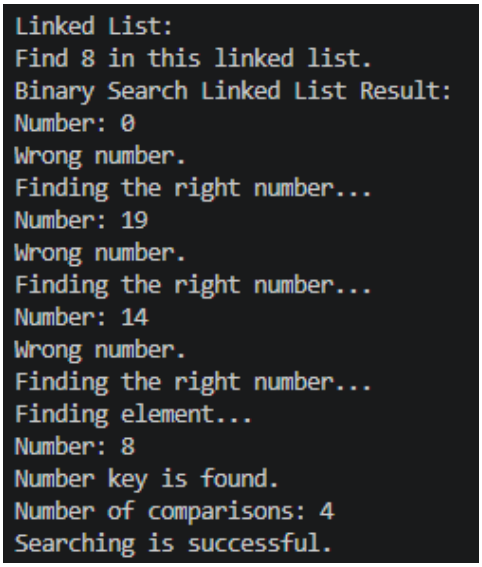
std::cout << "Find 8 in this linked list. \n";
std::cout << "Binary Search Linked List Result:\n";
suppleBinarySearchLLFind(Num1, 8);
```

Output

Array:	Linked List:



```
Problem 3
Array:
Finding the number 8.
Binary Search Array Result:
Number: 11
Wrong number.
Finding the right number...
Number: 5
Wrong number.
Finding the right number...
Number: 6
Wrong number.
Finding the right number...
Finding element...
Number: 8
Number key is found.
Number of comparisons: 4
Searching is successful.
```



```
Linked List:
Find 8 in this linked list.
Binary Search Linked List Result:
Number: 0
Wrong number.
Finding the right number...
Number: 19
Wrong number.
Finding the right number...
Number: 14
Wrong number.
Finding the right number...
Finding element...
Number: 8
Number key is found.
Number of comparisons: 4
Searching is successful.
```

Table 6.4c Problem #3

Problem	Problem 4. Modify the binary search algorithm so that the algorithm becomes recursive. Using this new recursive binary search, implement a solution to the same problem for problem 3.
Code	?
Output	?

Table 6.4d Problem #4

Conclusion:

I didn't finish the activity due to problem #4 since I was confused about recursion. Although I finished problems #1-#3 in the supplementary. Even though I wasn't able to finish the activity, I did learn how linear search and binary search on both arrays and linked list works.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: **Used to generate ideas to explain the codes in the procedure.**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used Gemini to generate ideas for the functions in the procedure to be able to make the observations better but all sentences are mine only.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.